

Functions to be done

Q1

- datasets/_loaders
 - load_spambase
 - load_fashion_mnist
- datasets/preprocessing

- _polynomial.py
- _gaussian.py

Q2

linear_models/classification/_logistic.py

Q3

model_selection/_split

- train_test_split

model_selection/_kfold

- class KFold

run_spam_experiment.py

Q4

Add operation given as reference (with broadcasting)

Can support broadcasting depending upon your convenience but broadcasting is one of the things that makes numpy powerful

linear

- call

containers

- call

activations.py

- relu
- sigmoid

-- visualize.py

- get_all_nodes_and_edges

(Logic of draw_dot has been given which draws computational graph using graphviz, but you are open to try anything else)

Q5

- model_selection/_split
train_test_val_split

Saving and Loading Models

For saving nn based models

(Model saving and loading logic has been given to you for your convenience in nn/modules/base
Can save by model.save_state_dict(save_path)

Can load by another instance of model with same architecture and then run
model2.load_state_dict(save_path)

For any other like binary class logistic regression

- Can use Pickle

(We didnt use Pickle for nn based because of our custom implementation but you can make and modify the logic to the logic that pickle expects.. (Your choice))

Other Key points

- In Q5 , the best performing model must be submitted with instructions to how to use it . It would be evaluated against a hidden dataset
- It's suggested not to use mnist test dataset as final hidden dataset is made by modifying it , using it for tuning might make your model generalize bad
- In Q4, you should reproduce a good theorticial justification of results
- For Q5, try to make it as short as possible (:)) Using your library
- SpamBase Dataset Link (Thought it exists in Data folder for your reference)
<https://archive.ics.uci.edu/dataset/94/spambase> SpamBase Dataset .
- For Q5 , refer
- https://colab.research.google.com/drive/1REudoW3X8RNA53vf8j_pir540JyzB7uR?usp=sharing
- https://www.youtube.com/watch?v=6EJaHBJhwDs&list=PLKnIA16_Rmvboy8bmDCjwNHgTaYH2puK7&index=7

Above is pytorch pipeline on the same dataset

With your library you can almost do the same!!!!

-the only key piece remaining is dataloader

For this assignment you can simply use create_batch function which spilts the data into batches and yields the result , but interested ones can try implementing dataloader and make your library 100% complete :)

- For Q5, one of the main things to learn is hyperparameter tuning . Though even base parameters

achieve 83% .. getting higher is what's the key is and most difficult part

You can do hyperparameter tuning yourselves.. using your logic , or you may use standard production libraries for tuning like Optuna

Check out this once

https://www.youtube.com/watch?v=Y3s-wBBLj_o&list=PLKnIA16_Rmvboy8bmDCjwNHgTaYH2puK7&index=10&pp=iAQB

(But again, this is optional., its your choice, but it would be useful)

(But again, this is optional., its your choice, but it would be useful)

- For Q5, try to see all these 5 kinds of models, how much maximum they can.. and why is some better than others ,which have to be answered with theoretical justifications.

Just for information

The state-of-the-art accuracy on the Fashion-MNIST dataset is 99.65%, achieved by a convolutional neural network (CNN) that incorporates data augmentation. A relatively simple Convolutional Neural Network (CNN) was used, called CNN-3-128, which featured three convolutional layer

<https://www.mdpi.com/2227-7390/12/20/3174>

As a just curiosity and to make you feel nice, you can achieve that too, the only thing is you need to support convolution operation in autograd.. and a convolution layer in nn/modules ,the backprop your autograd would take care. This we would study in depth in the course of Deep Learning .